WEEK 2 LEARNING

Topics Covered:

1. Git & GitHub Basics
2. JavaScript ES6: Arrow Functions and Arrays
3. Async Fetch API
4. DOM and Form Handling

1. Git & GitHub Basics

Git is a version control tool that runs on my computer. It keeps a history of changes to my project, so I can go back to an older version or work on new features without breaking the working code. **GitHub** is a website that hosts Git repositories online. It lets me back up my code and share it with other people.

Key terms

- **Repository (repo):** a project folder that Git is tracking, along with its full history.
- **Commit:** a saved snapshot of my changes, with a short message describing what changed.
- **Push:** sends my local commits to the remote repository on GitHub.
- **Pull:** brings new commits from GitHub and merges them into my current branch.
- **Fetch:** downloads new commits from GitHub but does not change my working files.

Basic workflow

The usual flow is: change files, stage them, commit, then push.

```
git clone <repo-url>        # copy a remote repo to my computer
git status                  # see which files changed
git add .                   # stage all changes
git commit -m "Add login form"
git push origin main        # upload commits to GitHub
git pull origin main        # get the latest changes
git branch                  # list branches
git checkout -b new-feature # create and switch to a new branch
```

git fetch vs git pull

	git fetch	git pull
What it does	Downloads remote changes only	Downloads and merges them
My files	Not changed	Updated
Risk	Safe, I can review first	Can cause merge conflicts

In short, **git pull = git fetch + git merge**. Fetch is useful when I just want to see what changed before touching my own code.

2. JavaScript ES6: Arrow Functions and Arrays

Arrow functions

Arrow functions are a shorter way to write functions. They use **=>** instead of the **function** keyword. If the body is a single expression, the **return** is implicit.

```
// Normal function
function add(a, b) {
  return a + b;
}

// Arrow function
const add = (a, b) => a + b;

// One parameter: brackets are optional
const double = n => n * 2;
```

Array methods

These three methods loop over an array and give back a result, without me writing a manual **for** loop. None of them change the original array.

`map()` **- transforms every item**

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8]
```

`filter()` **- keeps only items that pass a test**

```
const nums = [1, 2, 3, 4, 5, 6];
const evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6]
```

`reduce()` **- combines all items into a single value**

```
const prices = [100, 250, 50];
const total = prices.reduce((sum, price) => sum + price, 0);
console.log(total); // 400
```

In **reduce()**, **sum** is the running total and **0** is its starting value.

When they are useful

- **map()**: turning a list of data into a list of UI items, e.g. names into list elements.
- **filter()**: search results, or showing only completed tasks.
- **reduce()**: totals, such as the sum of a cart.

They are very common in React, where lists are usually rendered with **map()**.

3. Async Fetch API

An **API** (Application Programming Interface) lets one program ask another for data. A website often calls a web API, which replies with data, usually in **JSON** format. The **fetch()** function is the built-in way for JavaScript in the browser to make these requests.

Promises, async and await

Getting data from a server takes time, and JavaScript should not freeze while waiting. So **fetch()** returns a **Promise**, which is an object that says "the result will arrive later". A promise can be pending, fulfilled, or rejected.

- **async** placed before a function means the function works with promises.
- **await** pauses inside that function until the promise finishes, so the code reads top to bottom like normal code.

GET request, JSON and error handling

- A GET request asks the server for data; it is the default for **fetch()**.
- **response.json()** converts the response body into a JavaScript object. It is also a promise, so it needs **await**.
- **response.ok** is false for HTTP errors like 404 or 500. **fetch()** does not throw on these, so I check it myself.
- **try/catch** handles real failures such as no internet connection.

```
async function getUser() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1");

    if (!response.ok) {
      throw new Error("Request failed: " + response.status);
    }

    const data = await response.json();
    console.log(data.name);
  } catch (error) {
    console.error("Something went wrong:", error.message);
  }
}

getUser();
```

This example uses JSONPlaceholder, a free public API that is meant for practice.

4. DOM and Form Handling

The **DOM** (Document Object Model) is the browser's tree-like representation of an HTML page. JavaScript can use it to find elements, change text, update styles, and react to what the user does.

Key pieces

- **querySelector()** selects the first element that matches a CSS selector, e.g. **"#name"** for an id or **".card"** for a class.
- **addEventListener()** runs a function when an event happens, such as **click** or **submit**.
- **input.value** gives the text currently typed in an input field.
- **event.preventDefault()** stops the form from its default behaviour of reloading the page on submit. This lets JavaScript handle the data instead.

Example: a simple greeting form

```
<form id="greetForm">
  <input type="text" id="nameInput" placeholder="Enter your name" />
  <button type="submit">Greet</button>
</form>
<p id="result"></p>

<script>
  const form = document.querySelector("#greetForm");
  const nameInput = document.querySelector("#nameInput");
  const result = document.querySelector("#result");

  form.addEventListener("submit", (event) => {
    event.preventDefault();               // stop page reload
    const name = nameInput.value.trim();  // read the value

    if (name === "") {
      result.textContent = "Please enter your name.";
    } else {
      result.textContent = "Hello, " + name + "!";
    }
  });
</script>
```

When the form is submitted, the code reads the name, checks that it is not empty, and shows a message in the paragraph below. I used **textContent** rather than **innerHTML** because it treats the input as plain text, which is safer.